

Inventory Service Hierarchy & Build Precedence

A guide to the natural order in which the pieces of an inventory management system should be built so that each layer rests on something solid underneath. This document is about general inventory-application principles, not about any specific implementation.

1. The principle: build from the nouns up

Every inventory system, from a corner-shop spreadsheet to SAP, eventually answers four questions:

1. WHAT is the thing? (the item)
2. WHERE is it? (the place)
3. HOW MUCH is there? (the quantity)
4. WHY did the quantity change? (the movement)

You cannot answer "how much" until you have defined the item and the place. You cannot record a movement until you can measure quantity. So the build order is dictated by the question order: nouns first, then state, then events, then workflows on top of events, then analytics on top of workflows. Skipping a layer is the single most common reason inventory projects feel disconnected.

2. The five tiers of an inventory system

Tier 0 — Platform foundation

Nothing inventory-specific. These exist so the rest can be built safely.

- Identity and authorization (who is the user, what may they touch).
- Service registry / API gateway (so clients have one address).
- A logging and health-check standard every service obeys.

Proficiency check: a developer can log in, hit a protected endpoint, and see a heartbeat for every running service. Nothing about stock yet.

Tier 1 — Master data (the nouns)

These are the reference tables every other service reads. They rarely change but everything depends on them. Build them first, build them well, and freeze the schema before moving on.

- Item / product catalog — SKUs, variants, categories, attributes. Without a SKU id you literally cannot say anything else in the system.
- Units of measure — eaches, cases, pallets, conversion factors, barcodes. Quantity is meaningless without a unit.
- Locations — warehouses, zones, bins, capacity rules. "Where" needs an addressable hierarchy.
- Business partners — suppliers and customers. Movements always have a counterparty.

Proficiency check: you can create a SKU, define that one case = 24 eaches, and assign it a home bin in a warehouse — all through the UI, all persisted, all queryable.

Tier 2 — State of the world (the quantities)

Now that the nouns exist, you can record how many of each are where. This tier is small in surface area but is the heart of the whole system.

- Stock-on-hand per SKU per location.
- Lot and serial tracking for items that need it.
- Reservations — quantity that is physically present but already promised.
- A movement ledger — the append-only log that explains every change in stock.

A rule worth tattooing on your wrist: every quantity change in the entire system, no matter which workflow caused it, must produce exactly one ledger row. If a workflow updates stock without writing the ledger, you have just built a bug.

Proficiency check: you can pick any SKU at any warehouse, see the current quantity, and reach a screen that lists every event that contributed to that number — with timestamps, users, and reasons.

Tier 3 — Workflows (the verbs)

Only now do you build the operational workflows. Each workflow is a structured way of producing ledger rows. They split into inbound, outbound, and internal:

Inbound:

- Procurement / purchasing — purchase orders, approvals, supplier invoices.
- Receiving — ASN, goods receipt, putaway, discrepancy handling, QA.

Outbound:

- Sales / demand capture — sales orders, allocations, backorders.
- Fulfillment — pick, pack, ship, carrier handoff, tracking.

Internal:

- Stock movement — transfers between warehouses, adjustments, write-offs.

- Audit / cycle counting — count sessions, variances, approvals, corrections.

Build inbound before outbound. You cannot ship what you have not received. And build internal movement before audit, because audit produces movements as its output.

Proficiency check: a buyer can drive a SKU from "ordered" to "on the shelf", a salesperson can drive that same SKU from "on the shelf" to "out the door", and the ledger tells the whole story without anyone touching the database directly.

Tier 4 — Intelligence on top

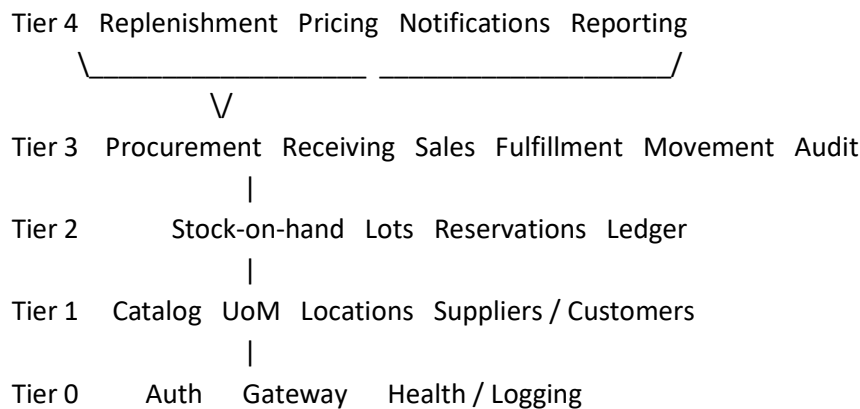
Once Tier 3 generates a steady stream of real events, the higher-value services finally have something to chew on. Building these earlier is wasted effort because they will have no real data to act on.

- Replenishment — reorder points, forecasting, suggested POs. Needs movement history.
- Pricing — cost rollups, margin analysis, discount rules. Needs purchase history and SKUs.
- Notifications — low-stock alerts, expiry warnings, exception emails. Needs events to react to.
- Reporting / analytics — KPIs, turns, aging, valuation. Needs the full ledger.

Proficiency check: the system can wake up on its own. It notices low stock and suggests a PO. It notices an expiring lot and emails a manager. It produces a daily KPI snapshot without a human pressing a button.

3. The dependency pyramid (visual summary)

Read bottom-up. Each tier consumes everything below it.



4. Hard precedence rules

5. No tier may be started until every service in the tier below it can be demonstrated end-to-end (not just compiled — demonstrated).

6. Within a tier, services that are pure read-targets (catalog, UoM, locations) come before services that write (procurement, sales). Reads stabilize the contract that writers depend on.
7. Inbound workflows precede outbound workflows. You must be able to put stock into the system before you can take it out.
8. Internal movement precedes audit. Audit corrections are themselves a kind of movement.
9. Analytics services come last. Building a dashboard before there is data to show is the oldest mistake in this domain.

5. Proficiency milestones — what "done" looks like at each step

Use these as gates. Do not move to the next tier until the current one passes.

Milestone	You can prove proficiency by doing this without code changes
Foundation up	Log in, see a live menu of services, every tile shows green, every action you take is in the audit log of who did what when.
Master data up	Create a brand new SKU, a brand new warehouse with three bins, a brand new supplier, and a unit of measure conversion — all in under five minutes through the UI.
State layer up	Manually push stock into a bin and watch the ledger record it. Run a SQL sum on the ledger and confirm it matches the on-hand display down to the unit.
Inbound up	Issue a PO, receive it partially, see the discrepancy, receive the rest, and observe stock rise step by step — every step traceable from PO to ledger.
Outbound up	Take a customer order for more than is in stock, watch part allocate and part backorder, pick and ship the allocated portion, watch the order line close itself.
Internal up	Transfer stock between two warehouses, run a cycle count that finds a variance, approve the adjustment, and confirm both warehouses are now consistent with reality.
Intelligence up	Leave the system alone for a day. Come back to a low-stock email, a suggested PO already drafted, and a KPI snapshot you did not generate by hand.

6. Common precedence mistakes

- Building dashboards for every service in parallel before any of them produce real data. The dashboards become hardcoded mock screens that quietly drift from the real model.
- Starting with reporting because it is "the visible part". Reporting is downstream of everything; building it first means building it twice.
- Skipping the ledger and updating `stock_on_hand` directly from each workflow. Looks faster for a week, then becomes impossible to debug forever.
- Treating UoM as an afterthought. Once any quantity is recorded without an explicit unit, the data is permanently ambiguous and migrations get painful.
- Wiring inter-service writes through synchronous REST calls before the receiving service is even built. Use events from day one.

7. The short version

Identity, then nouns, then quantities, then verbs, then intelligence. Refuse to skip a tier and refuse to call a tier finished until you can demonstrate it cold to someone who has never seen the system. Inventory software is unforgiving of shortcuts because every shortcut eventually shows up as a number that does not match reality — and reality always wins.